12 - Fechas, hilos y JVM

Pregunta número 1

¿Cuál es la salida del siguiente código?

```
import java.time.LocalDate;
import java.time.Month;

public class Test {
    public static void main(String[] argv) {
        LocalDate date = LocalDate.of(1997, Month.MAY, 50);
        System.out.println(date.getYear() + "-" + date.getMonth() + "-" + date.getDayOfMonth());
    }
}
```

Seleccione una:

- a. 1997-MAY-15
- b. 1997-JUN-15
- c. Se genera una excepción en tiempo de ejecución.
- d. El código no compila.
- e. 14-JUN-1997

Respuesta: En este ejercicio, si cometemos el error de leer rápido, podríamos ponernos a calcular a qué fecha "se recorren" los 50 días, pero la API de java.time no funciona así. El método LocalDate.of() valida estrictamente que la fecha exista en el calendario, y mayo solo tiene 31 días, así que pedir el día 50 es inválido. En lugar de ajustar la fecha al mes siguiente, como hacía la antigua clase Calendar en modo permisivo, LocalDate.of() lanza una DateTimeException en tiempo de ejecución, por lo que la línea del println nunca se alcanza. El código sí compila sin problemas, ya que el compilador solo revisa que los argumentos sean del tipo correcto (un int, un Month y otro int), y no puede saber que el 50 es un día inexistente; eso descarta el inciso d. Los incisos a, b y e son distractores que intentan hacernos pensar que Java corrige o recorre la fecha de alguna forma, cosa que nunca hace con LocalDate.of().

Pregunta número 2

What is the result?

```
public class Test {
    int[] array = {1, 2, 3, 4, 5};
    public static void main(String[] args) {
        new Test().go(1);
    }
    void go(int x) {
        array[x % 5] = x++;
        go(x);
    }
}
```

Seleccione una:

- a. A java.lang.ArrayIndexOutOfBoundsException is thrown.
- b. A java.lang.IllegalStateException is thrown.
- c. A java.lang.StackOverflowError is thrown.
- d. The program runs with no output.
- e. Compilation fails

Respuesta: El primer punto que me gustaría tocar es que el código compila sin problemas: array es un atributo de instancia y go() es un método de instancia que llamamos desde un objeto creado con new Test(). Al ejecutarlo, go(1) asigna x a la posición x % 5 del arreglo, incrementa x y se llama a sí mismo con el nuevo valor, y así indefinidamente. A mí lo primero que me preocuparía es el índice, pero el operador % 5 lo mantiene siempre entre 0 y 4, así que nunca salimos de los límites del arreglo y el inciso a queda descartado. El verdadero problema es que la recursión no tiene ninguna condición de paro. Cada llamada agrega un nuevo marco (frame) en la pila de ejecución del hilo, y como nunca se libera ninguno, la pila termina por llenarse y la JVM lanza un StackOverflowError. Debemos recordar que es un Error, no una Exception, porque es un problema de recursos de la JVM. El programa nunca termina normalmente, así que el inciso d tampoco es posible.

Pregunta número 3

What is the result?

```
import java.util.*;
public class MyScan {
    public static void main(String[] args) {
        String in = "1 a 10 . 100 1000";
        Scanner s = new Scanner(in);
        int accum = 0;
        for (int x = 0; x < 4; x++) {
            accum += s.nextInt();
        }
        System.out.println(accum);
    }
}
```

Seleccione una:

- a. 11
- b. 111
- c. 1111
- d. An exception is thrown at runtime.

Respuesta: En este ejercicio hay algo que me llamó la atención desde el principio y eso es que la variable se llama in, y aunque podría confundirnos con System.in, aquí es simplemente un String, así que el Scanner va a leer ese texto. Por defecto, Scanner separa el contenido usando los espacios en blanco, por lo que los tokens son "1", "a", "10", ".", "100" y "1000". El bucle llama cuatro veces a nextInt(). En la primera iteración, el token es "1", que sí es un entero válido, y accum pasa a valer 1. En la segunda iteración, el siguiente token es "a", y aquí está el problema: nextInt() espera un número entero y, al encontrar una letra, lanza una InputMismatchException. Esta excepción es unchecked y nadie la atrapa, así que el programa termina con la excepción y el println nunca se ejecuta. Los incisos a, b y c son trampas para quien piense que el Scanner ignora los tokens que no son números y suma solo 1, 10, 100 y 1000; nextInt() no salta nada, y si el token no es un entero, falla.

Pregunta número 4

What is the result?

```
public class Bees {
    public static void main(String[] args) {
        try {
            new Bees().go();
        } catch (Exception e) {
            System.out.println("thrown to main");
        }
    }
    synchronized void go() throws InterruptedException {
        Thread t1 = new Thread();
        t1.start();
        System.out.print("1 ");
        t1.wait(5000);
        System.out.print("2 ");
    }
}
```

Seleccione una:

- a. The program prints 1 then 2 after 5 seconds.
- b. The program prints: 1 thrown to main.
- c. The program prints: 1 2 thrown to main.
- d. The program prints:1 then t1 waits for its notification.

Respuesta: Siento que lo más importante de este ejercicio es entender qué monitor (lock) tiene el hilo en cada momento. El método go() es synchronized, así que el hilo principal adquiere el lock del objeto Bees sobre el que se llamó, es decir, el de this. Dentro del método se crea y se inicia t1, y se imprime "1 ". Luego viene t1.wait(5000). Aquí está la trampa: para llamar a wait(), notify() o notifyAll() sobre un objeto, el hilo debe tener el lock de ese mismo objeto. Nosotros tenemos el lock de this, no el de t1, por lo que la JVM lanza una IllegalMonitorStateException. Esta excepción es unchecked y hereda de RuntimeException, así que se propaga fuera de go(); el "2 " jamás se imprime y llega al main, donde el catch (Exception e) la

atrapa e imprime "thrown to main". La salida final es "1 thrown to main". El inciso a sería el resultado si el código llamara a this.wait(5000); el inciso c es imposible porque el "2 " nunca se alcanza, y el inciso d describe un comportamiento que no ocurre, porque t1 ni siquiera es quien espera.

Pregunta número 5

Which of the following will be the output? (Choose two.)

```
public class Test implements Runnable {
    boolean flag = true;
    @Override
    public synchronized void run() {
        if (flag) {
            flag = false;
            System.out.print("1 ");
            try {
                 this.wait();
            } catch (Exception e) {
            }
            System.out.print("2 ");
        } else {
            flag = true;
            System.out.print("3 ");
            try {
                Thread.sleep(2000);
            } catch (Exception e) {
            }
            System.out.print("4 ");
            notify();
        }
    }
    public static void main(String[] args) {
        Test w = new Test();
        new Thread(w).start();
        new Thread(w).start();
    }
}
```

Seleccione una o más de una:

- [ ] a. The code outputs 1 3 4.
- [ ] b. The code outputs 1 2 3 4.
- [ ] c. The code outputs 3 4 1.
- [x] d. The code outputs 1 3 4 2.
- [ ] e. The code never completes.
- [x] f. The code runs to completion.

Respuesta: Aquí tenemos dos hilos que comparten el mismo objeto Runnable w, por lo que comparten la variable flag y el mismo monitor. Como run() es synchronized, solo un hilo a la vez puede ejecutarlo. Supongamos que entra primero cualquiera de los dos; como flag vale true, toma la primera rama: cambia flag a false, imprime "1 " y llama a this.wait(). Un punto clave es que wait() libera el lock mientras el hilo espera. Gracias a eso, el segundo hilo puede entrar a run(); ahora flag es false, así que toma la rama else: cambia flag a true, imprime "3 ", duerme 2 segundos con Thread.sleep(2000) (que, a diferencia de wait(), no libera el lock), imprime "4 " y llama a notify(). Ese notify() despierta al primer hilo, pero este solo puede continuar cuando el segundo hilo sale del método synchronized y libera el lock. Entonces el primer hilo sale de su wait() e imprime "2 ". La salida es "1 3 4 2" y, como ambos hilos terminan, el programa corre hasta completarse. Los incisos a y c omiten el "2 " o alteran el orden; el inciso b pone el "2 " antes de que nadie haya hecho notify(), y el inciso e sería cierto solo si nadie despertara al hilo que espera, lo cual no ocurre aquí.

Pregunta número 6

What is the result?

```
import java.text.*;
public class Align {
    public static void main(String[] args) throws ParseException{
        String[] sa = {"111.234", "222.5678"};
        NumberFormat nf = NumberFormat.getInstance();
        nf.setMaximumFractionDigits(3);
        for (String s : sa){ System.out.println(nf.parse(s));}
    }
}
```

Seleccione una:

- a. 111.234 222.568
- b. 111.234 222.5678
- c. An exception is thrown at runtime.
- d. 111.234 222.567

Respuesta: En este ejercicio hay una trampa con la que debemos tener cuidado, se encuentra en el método setMaximumFractionDigits(3). Este método configura cuántos decimales se van a mostrar cuando formateamos un número, es decir, cuando convertimos un número a texto con format(). Pero en el código no se usa format(), sino parse(), que hace lo contrario: convierte un texto en un número. A parse() no le afecta el número máximo de decimales, así que lee los valores completos tal como vienen en el String. El primer texto se convierte en 111.234 y el

segundo en 222.5678, y cada uno se imprime con el toString() del Number que regresa parse(). Por eso la salida es 111.234 y 222.5678. Los incisos a y d serían la trampa para quien piense que el límite de 3 decimales se aplica al parsear, redondeando (222.568) o truncando (222.567). Tampoco hay ninguna excepción, porque ambos textos son números válidos, y el método main declara ParseException, así que el código compila.

Pregunta número 7

¿Cuál es la respuesta verdadera?

```
public class Test implements Runnable {
    private int i;
    @Override
    public void run() {
        if (i % 10 == 0) {
            i++;
        }
        for (int x = 0; x < 10; x++, i++) {
            if (x == 4) {
                Thread.yield();
            }
            System.out.println(i + " ");
        }
    }
    public static void main(String[] args) {
    Test n = new Test();
    for (int x = 0; x < 100; x++) {
        new Thread(n).start();
    }
 }
}
```

Seleccione una:

- a. La salida nunca puede contener el valor 297.
- b. La salida nunca puede contener el valor 1020.
- c. La salida nunca puede contener el valor 1130.
- d. La salida nunca puede contener el valor 30.

Respuesta: Este ejercicio es más de razonamiento que de cálculo, porque la variable i es compartida por los 100 hilos y no hay ninguna sincronización, así que la salida exacta es impredecible. Por eso, en lugar de buscar el resultado, nos conviene calcular el valor máximo que i podría alcanzar. Cada hilo, al ejecutar run(), hace como máximo 11 incrementos: uno dentro del if, si i % 10 == 0, y diez en la

parte de actualización del for (x++, i++). Si multiplicamos por los 100 hilos, i puede incrementarse como mucho 1,100 veces, partiendo de 0. Además, por las condiciones de carrera (race conditions), algunos incrementos se pueden perder, pero nunca se pueden "duplicar", así que i jamás podrá pasar de 1100. Por lo tanto, el valor 1130 nunca podrá aparecer en la salida. Los otros valores sí son posibles: 30 y 297 están dentro del rango normal, y 1020 también puede aparecer, porque el if puede ejecutarse más veces de lo esperado cuando varios hilos leen i justo en un múltiplo de 10. De hecho, si ejecutamos el programa varias veces, en algunas corridas el valor máximo impreso llega a ser mayor a 1000, lo que confirma que se puede rebasar ese número, pero nunca el límite de 1100.

Pregunta número 8

Assume that the default locale formats FormatStyle.SHORT date-times as dd/MM/yy HH:mm. What is the result?

```
import java.time.LocalDateTime;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;

public class Test {
    public static void main(String[] argv) {
        LocalDateTime d = LocalDateTime.of(1997, 5, 15, 23, 55, 30);
        Period p = Period.of(1, 2, 3);
        d = d.minus(p);
        DateTimeFormatter f =
            DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);
        System.out.println(d.format(f));
    }
}
```

Seleccione una:

- a. 15/05/1997
- b. The code does not compile
- c. 05/15/97 23:55 AM
- d. 0.996875
- e. A runtime exception is thrown
- f. 12/3/1996
- g. 12/03/96 23:55

Respuesta: En este ejercicio hay que ir paso a paso para entender qué está sucediendo. Primero creamos el LocalDateTime con el 15 de mayo de 1997 a las 23:55:30. Después restamos un Period.of(1, 2, 3), que representa 1 año, 2 meses y

3 días. Las clases de java.time son inmutables, pero en este caso sí reasignamos el resultado a la variable d, así que el cambio se conserva. La resta se aplica en orden: primero el año (15 de mayo de 1996), después los meses (15 de marzo de 1996) y finalmente los días, con lo que llegamos al 12 de marzo de 1996. La hora no cambia, porque un Period solo trabaja con fechas, así que seguimos teniendo las 23:55:30. Luego creamos un formateador con ofLocalizedDateTime(FormatStyle.SHORT), que es válido para un LocalDateTime y no lanza excepción; solo los estilos LONG y FULL fallarían, porque necesitan una zona horaria. Como la pregunta nos dice que el locale formatea SHORT como dd/MM/yy HH:mm, el resultado es "12/03/96 23:55". El inciso f omite la hora y usa el año completo, y el inciso c no aplica el Period y usa el formato de Estados Unidos.